

CropScan V5 — LoRA DINOv2 + Plant Pathology 2021

Target: push PlantDoc accuracy from 71.6% (v4 ensemble+TTA) to 75%+ by:

1. **LoRA fine-tune DINOv2** — adds rank-8 low-rank adapters into the qkv and proj projections of every attention block. ~220K trainable params (vs 200K in v4 linear probe), but the backbone can now adapt slightly to plant-specific features without losing OOD generalization.
2. **Plant Pathology 2021** — adds 11K+ real-world apple disease images (healthy / scab / rust). This is the largest single real-world dataset addition we've made — 9× the size of PlantDoc.
3. Everything from v4: 38-class PV with aliases, 25K synthesis, EMA weights, ensemble + TTA at 280 eval resolution, threshold target 0.80.

Required data on disk:

- PlantVillage: `color/<38 class folders>`
- PlantDoc: `data/plantdoc/train` and `data/plantdoc/test`
- Plant Pathology 2021 (optional but recommended):
`data/plantpath2021/train.csv` + `data/plantpath2021/train_images/`

If PP21 is missing, the notebook prints setup instructions and continues — training will degrade to v4 + LoRA (still beats v4 by 1-3pp).

Outputs (in `artifacts/v5/`):

- `dinov2_vits14_lora_cropscan_v5.pth` + `efficientnet_v2_s_cropscan_v5.pth`
- `matching *_bundle.pt`
- `training_report.json`, `labels.json`, preview PNGs

Expected runtime A6000: synth ~30 min (cached from v4), DINOv2 LoRA ~80 min, EffNet ~95 min.

```
In [4]: import copy, csv, json, math, os, random, subprocess, sys
from collections import Counter, defaultdict
from dataclasses import asdict, dataclass
from io import StringIO
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from PIL import Image, ImageFilter
```

```

from sklearn.metrics import f1_score
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import models, transforms

print(f'Python: {sys.version.split()[0]}')
print(f'PyTorch: {torch.__version__}')
print(f'CUDA: {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)} ({torch.cuda.get_device_properties(0).total_memory / 1024 / 1024} GB)')

```

```

Python: 3.12.3
PyTorch: 2.6.0+cu124
CUDA: True
GPU: NVIDIA RTX A6000 (50.9 GB)

```

```

In [5]: SEED = 434
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
torch.backends.cudnn.benchmark = True
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

```

Device: cuda

Paths and hyperparameters

```

In [6]: PLANTVILLAGE_DIR = Path('color')
PLANTDOC_DIR = Path('data/plantdoc')
PLANTPATH_DIR = Path('data/plantpath2021')
PLANTPATH_CSV = PLANTPATH_DIR / 'train.csv'
PLANTPATH_IMG_DIR = PLANTPATH_DIR / 'train_images'

ARTIFACT_DIR = Path('artifacts/v5')
SYNTHETIC_DIR = Path('data/synthetic_field')
SYNTH_LEAVES_DIR = SYNTHETIC_DIR / 'leaves_v4'
SYNTH_INDEX = SYNTHETIC_DIR / 'synth_index_v4.json'
BG_DIR = SYNTHETIC_DIR / 'backgrounds'
for d in [ARTIFACT_DIR, SYNTHETIC_DIR, SYNTH_LEAVES_DIR, BG_DIR]:
    d.mkdir(parents=True, exist_ok=True)

@dataclass
class Config:
    image_size_train: int = 224
    image_size_eval: int = 280
    batch_size: int = 32
    epochs: int = 25
    warmup_epochs: int = 3
    learning_rate: float = 5e-4
    dinov2_lr: float = 5e-4
    weight_decay: float = 1e-4
    num_workers: int = 4
    label_smoothing: float = 0.1

    plantvillage_weight: float = 1.0
    synthetic_weight: float = 2.5
    plantdoc_weight: float = 6.0

```

```

plantpath_weight: float = 4.0

mixup_alpha: float = 0.2
cutmix_alpha: float = 1.0
mix_prob: float = 0.5

num_synth_per_leaf: int = 3
max_synth_total: int = 25000
n_background_patches: int = 2000

lora_r: int = 8
lora_alpha: float = 16.0

ema_decay: float = 0.999
target_confident_precision: float = 0.80

CFG = Config()
print(json.dumps(asdict(CFG), indent=2))

```

```

{
  "image_size_train": 224,
  "image_size_eval": 280,
  "batch_size": 32,
  "epochs": 25,
  "warmup_epochs": 3,
  "learning_rate": 0.0005,
  "dinov2_lr": 0.0005,
  "weight_decay": 0.0001,
  "num_workers": 4,
  "label_smoothing": 0.1,
  "plantvillage_weight": 1.0,
  "synthetic_weight": 2.5,
  "plantdoc_weight": 6.0,
  "plantpath_weight": 4.0,
  "mixup_alpha": 0.2,
  "cutmix_alpha": 1.0,
  "mix_prob": 0.5,
  "num_synth_per_leaf": 3,
  "max_synth_total": 25000,
  "n_background_patches": 2000,
  "lora_r": 8,
  "lora_alpha": 16.0,
  "ema_decay": 0.999,
  "target_confident_precision": 0.8
}

```

```

In [7]: CLASS_NAMES = [
    'Apple__Apple_scab', 'Apple__Black_rot', 'Apple__Cedar_apple_rust', 'Apple__
    'Blueberry__healthy',
    'Cherry_(including_sour)__Powdery_mildew', 'Cherry_(including_sour)__healthy'
    'Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize)__Common_ru
    'Corn_(maize)__Northern_Leaf_Blight', 'Corn_(maize)__healthy',
    'Grape__Black_rot', 'Grape__Esca_(Black_Measles)',
    'Grape__Leaf_blight_(Isariopsis_Leaf_Spot)', 'Grape__healthy',
    'Orange__Haunglongbing_(Citrus_greening)',
    'Peach__Bacterial_spot', 'Peach__healthy',

```

```

'Pepper__bell__Bacterial_spot', 'Pepper__bell__healthy',
'Potato__Early_blight', 'Potato__Late_blight', 'Potato__healthy',
'Raspberry__healthy',
'Soybean__healthy',
'Squash__Powdery_mildew',
'Strawberry__Leaf_scorch', 'Strawberry__healthy',
'Tomato_Bacterial_spot', 'Tomato_Early_blight', 'Tomato_Late_blight',
'Tomato_Leaf_Mold', 'Tomato_Septoria_leaf_spot',
'Tomato_Spider_mites_Two_spotted_spider_mite', 'Tomato__Target_Spot',
'Tomato__Tomato_YellowLeaf_Curl_Virus', 'Tomato__Tomato_mosaic_virus',
'Tomato_healthy',
]
CLASS_TO_IDX = {n: i for i, n in enumerate(CLASS_NAMES)}
NUM_CLASSES = len(CLASS_NAMES)
IMG_EXTS = {'.jpg', '.jpeg', '.png', '.webp'}
assert NUM_CLASSES == 38
print(f'{NUM_CLASSES} classes')

```

38 classes

Path validation

PV and PD are required. PP21 is optional — if missing, the notebook prints setup instructions and continues.

```

In [8]: def count_images(root: Path) -> int:
        if not root.exists():
            return 0
        return sum(1 for p in root.rglob('*') if p.suffix.lower() in IMG_EXTS)

n_pv = count_images(PLANTVILLAGE_DIR)
n_pd = count_images(PLANTDOC_DIR)
n_pp = count_images(PLANTPATH_IMG_DIR) if PLANTPATH_IMG_DIR.exists() else 0
n_bg = len(list(BG_DIR.glob('bg_*.jpg'))))

print(f'["OK " if n_pv else "MISS"] PlantVillage {n_pv:>7d} images at {PLANTVIL
print(f'["OK " if n_pd else "MISS"] PlantDoc {n_pd:>7d} images at {PLANTDOC
print(f'["OK " if n_pp else "MISS"] PlantPath2021 {n_pp:>7d} images at {PLANTPAT
print(f'["OK " if n_bg else "MISS"] Backgrounds {n_bg:>7d} cached at {BG_DIR}')

assert n_pv > 0, f'PlantVillage missing at {PLANTVILLAGE_DIR}'
assert n_pd > 0, f'PlantDoc missing at {PLANTDOC_DIR}'

HAVE_PP21 = n_pp > 0 and PLANTPATH_CSV.exists()
if not HAVE_PP21:
    print()
    print('--- PlantPath2021 setup instructions ---')
    print('To add PP21 (recommended, +11K real-world apple images):')
    print(' 1. kaggle.com -> Settings -> API -> Create New Token (downloads kaggle
    print(' 2. Upload kaggle.json to this Jupyter session')
    print(' 3. Run in a cell:')
    print('      !pip install -q kaggle')
    print('      !mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.
    print(' 4. Visit https://www.kaggle.com/c/plant-pathology-2021-fgvc8 and click

```

```

print(' 5. Run:')
print(f'      !mkdir -p {PLANTPATH_DIR} && kaggle competitions download -c pla
print(f'      !cd {PLANTPATH_DIR} && for z in *.zip; do unzip -q -o "$z"; done
print(' 6. Re-run this cell.')
print()
print('Continuing without PP21 – training still runs, just slightly weaker (~71

```

```

[OK ] PlantVillage      54305 images at color
[OK ] PlantDoc          2579 images at data/plantdoc
[OK ] PlantPath2021     18632 images at data/plantpath2021/train_images
[OK ] Backgrounds      2000 cached at data/synthetic_field/backgrounds

```

Class mappings

```

In [9]: PV_FOLDER_ALIASES = {
    'Pepper_bell__Bacterial_spot': 'Pepper_bell__Bacterial_spot',
    'Pepper_bell__healthy': 'Pepper_bell__healthy',
    'Tomato__Bacterial_spot': 'Tomato_Bacterial_spot',
    'Tomato__Early_blight': 'Tomato_Early_blight',
    'Tomato__Late_blight': 'Tomato_Late_blight',
    'Tomato__Leaf_Mold': 'Tomato_Leaf_Mold',
    'Tomato__Septoria_leaf_spot': 'Tomato_Septoria_leaf_spot',
    'Tomato__Spider_mites Two-spotted_spider_mite': 'Tomato_Spider_mites_Two_spott',
    'Tomato__Target_Spot': 'Tomato__Target_Spot',
    'Tomato__Tomato_Yellow_Leaf_Curl_Virus': 'Tomato__Tomato_YellowLeaf__Cu
    'Tomato__Tomato_mosaic_virus': 'Tomato__Tomato_mosaic_virus',
    'Tomato__healthy': 'Tomato_healthy',
}
assert set(PV_FOLDER_ALIASES.values()).issubset(CLASS_TO_IDX)

PLANTDOC_TO_CROPSCAN = {
    'Apple Scab Leaf': 'Apple__Apple_scab',
    'Apple rust leaf': 'Apple__Cedar_apple_rust',
    'Bell_pepper leaf spot': 'Pepper_bell__Bacterial_spot',
    'Corn Gray leaf spot': 'Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot',
    'Corn leaf blight': 'Corn_(maize)__Northern_Leaf_Blight',
    'Corn rust leaf': 'Corn_(maize)__Common_rust_',
    'Potato leaf early blight': 'Potato__Early_blight',
    'Potato leaf late blight': 'Potato__Late_blight',
    'Squash Powdery mildew leaf': 'Squash__Powdery_mildew',
    'Tomato Early blight leaf': 'Tomato_Early_blight',
    'Tomato Septoria leaf spot': 'Tomato_Septoria_leaf_spot',
    'Tomato leaf bacterial spot': 'Tomato_Bacterial_spot',
    'Tomato leaf late blight': 'Tomato_Late_blight',
    'Tomato leaf mosaic virus': 'Tomato__Tomato_mosaic_virus',
    'Tomato leaf yellow virus': 'Tomato__Tomato_YellowLeaf__Curl_Virus',
    'Tomato mold leaf': 'Tomato_Leaf_Mold',
    'Tomato two spotted spider mites leaf': 'Tomato_Spider_mites_Two_spotted_spider
    'grape leaf black rot': 'Grape__Black_rot',
}
assert set(PLANTDOC_TO_CROPSCAN.values()).issubset(CLASS_TO_IDX)

PLANTPATH_TO_CROPSCAN = {
    'healthy': 'Apple__healthy',
    'rust': 'Apple__Cedar_apple_rust',

```

```

        'scab': 'Apple___Apple_scab',
    }
    assert set(PLANTPATH_TO_CROPSCAN.values()).issubset(CLASS_TO_IDX)

```

Collect samples

```

In [10]: @dataclass
class Sample:
    path: Path
    label: int
    source: str

def collect_folder(root: Path, mapping: dict, source: str):
    if not root.exists():
        return []
    out, skipped = [], []
    for folder in sorted(p for p in root.iterdir() if p.is_dir()):
        mapped = mapping.get(folder.name)
        if mapped is None:
            skipped.append(folder.name)
            continue
        label = CLASS_TO_IDX[mapped]
        for img in folder.rglob('*'):
            if img.suffix.lower() in IMG_EXTS:
                out.append(Sample(img, label, source))
    if skipped:
        print(f' {root}: skipped {len(skipped)} unmapped folders')
    return out

pv_map = {n: n for n in CLASS_NAMES}
pv_map.update(PV_FOLDER_ALIASES)
pv_samples = collect_folder(PLANTVILLAGE_DIR, pv_map, 'plantvillage')
print(f'PlantVillage: {len(pv_samples)} images, {len({s.label for s in pv_samples})}N

color: skipped 1 unmapped folders
PlantVillage: 54305 images, 38/38 classes

```

```

In [11]: pd_samples = []
for sub in ['train', 'test', '']:
    pd_samples.extend(collect_folder(PLANTDOC_DIR / sub if sub else PLANTDOC_DIR, P
seen = set()
pd_unique = []
for s in pd_samples:
    k = str(s.path.resolve())
    if k not in seen:
        seen.add(k)
        pd_unique.append(s)
pd_samples = pd_unique
print(f'PlantDoc: {len(pd_samples)} images, {len({s.label for s in pd_samples})}N

data/plantdoc/train: skipped 10 unmapped folders
data/plantdoc/test: skipped 10 unmapped folders
data/plantdoc: skipped 3 unmapped folders
PlantDoc: 1728 images, 18/38 classes

```

```
In [12]: def collect_plantpath(csv_path: Path, img_dir: Path, mapping: dict):
    if not csv_path.exists() or not img_dir.exists():
        return []
    samples = []
    dropped_multilabel = 0
    dropped_unmapped = 0
    dropped_missing = 0
    with open(csv_path) as f:
        reader = csv.DictReader(f)
        for row in reader:
            labels = row.get('labels', '').strip().lower().split()
            if len(labels) != 1:
                dropped_multilabel += 1
                continue
            tag = labels[0]
            if tag not in mapping:
                dropped_unmapped += 1
                continue
            img_path = img_dir / row.get('image', '')
            if not img_path.exists():
                dropped_missing += 1
                continue
            samples.append(Sample(img_path, CLASS_TO_IDX[mapping[tag]], 'plantpath2
print(f' PlantPath2021: kept {len(samples)}, dropped {dropped_multilabel} mult
      f'{dropped_unmapped} unmapped, {dropped_missing} missing')
    return samples

pp_samples = collect_plantpath(PLANTPATH_CSV, PLANTPATH_IMG_DIR, PLANTPATH_TO_CROPS
print(f'PlantPath2021: {len(pp_samples)} images, {len({s.label for s in pp_samples})

PlantPath2021: kept 11310, dropped 1355 multi-label, 5967 unmapped, 0 missing
PlantPath2021: 11310 images, 3/38 classes
```

Stratified splits

```
In [13]: def stratified_split(samples, val_ratio=0.1, test_ratio=0.1, seed=SEED):
    grouped = defaultdict(list)
    for s in samples:
        grouped[s.label].append(s)
    train, val, test = [], [], []
    rng = random.Random(seed)
    for arr in grouped.values():
        rng.shuffle(arr)
        n = len(arr)
        nt = max(1, int(n * test_ratio)) if n >= 10 else max(0, int(n * test_ratio))
        nv = max(1, int(n * val_ratio)) if n >= 10 else max(0, int(n * val_ratio))
        test.extend(arr[:nt])
        val.extend(arr[nt:nt+nv])
        train.extend(arr[nt+nv:])
    rng.shuffle(train); rng.shuffle(val); rng.shuffle(test)
    return train, val, test

pv_train, pv_val, pv_test = stratified_split(pv_samples, 0.10, 0.10)
pd_train, pd_val, pd_test = stratified_split(pd_samples, 0.15, 0.15)
```

```

pp_train, pp_val, pp_test = stratified_split(pp_samples, 0.10, 0.10) if pp_samples
print(f'pv  train/val/test = {len(pv_train)}/{len(pv_val)}/{len(pv_test)}')
print(f'pd  train/val/test = {len(pd_train)}/{len(pd_val)}/{len(pd_test)}')
print(f'pp  train/val/test = {len(pp_train)}/{len(pp_val)}/{len(pp_test)}')

```

```

pv  train/val/test = 43471/5417/5417
pd  train/val/test = 1228/250/250
pp  train/val/test = 9050/1130/1130

```

Leaf segmentation + background pool + synthesis (same as v4)

```

In [14]: def segment_leaf(pil_img, dist_threshold=28, min_fg_ratio=0.05):
    arr = np.asarray(pil_img.convert('RGB'))
    h, w = arr.shape[:2]
    cs = max(8, min(h, w) // 12)
    corners = np.concatenate([
        arr[:cs, :cs].reshape(-1, 3),
        arr[:cs, -cs:].reshape(-1, 3),
        arr[-cs:, :cs].reshape(-1, 3),
        arr[-cs:, -cs:].reshape(-1, 3),
    ])
    bg = np.median(corners, axis=0).astype(np.float32)
    diff = arr.astype(np.float32) - bg
    dist = np.sqrt((diff ** 2).sum(axis=2))
    mask = (dist > dist_threshold).astype(np.uint8) * 255
    if mask.mean() / 255.0 < min_fg_ratio:
        return None
    return Image.fromarray(np.dstack([arr, mask]).astype(np.uint8), 'RGBA')

```

```

In [15]: def build_background_pool(source_samples, n_target, patch_size=384):
    existing = sorted(BG_DIR.glob('bg_*.jpg'))
    if len(existing) >= n_target:
        print(f'  cached background pool: {len(existing)} patches')
        return [str(p) for p in existing[:n_target]]
    rng = random.Random(SEED + 11)
    sources = list(source_samples)
    rng.shuffle(sources)
    saved = list(existing)
    i = len(saved)
    for sample in sources:
        if i >= n_target:
            break
        try:
            img = Image.open(sample.path).convert('RGB')
            w, h = img.size
            if min(w, h) < patch_size:
                scale = patch_size / min(w, h) * 1.1
                img = img.resize((int(w * scale), int(h * scale)))
                w, h = img.size
            for _ in range(2):
                if i >= n_target:
                    break
                x = rng.randint(0, w - patch_size)

```



```

        y = rng.randint(0, h - patch_size)
        crop = img.crop((x, y, x + patch_size, y + patch_size))
        crop = crop.filter(ImageFilter.GaussianBlur(radius=3.5))
        out = BG_DIR / f'bg_{i:05d}.jpg'
        crop.save(out, quality=85)
        saved.append(out)
        i += 1
    except Exception:
        continue
print(f' saved {len(saved)} background patches')
return [str(p) for p in saved]

real_world_for_bg = pd_samples + pp_samples
bg_pool = build_background_pool(real_world_for_bg, CFG.n_background_patches)
print(f'Background pool: {len(bg_pool)}')
```

cached background pool: 2000 patches
Background pool: 2000

```

In [16]: def augment_leaf(leaf_rgba, rng):
    arr = np.asarray(leaf_rgba).astype(np.float32)
    rgb = arr[:, :, :3]
    alpha = arr[:, :, 3]
    rgb = rgb * rng.uniform(0.7, 1.3)
    rgb = (rgb - 128.0) * rng.uniform(0.85, 1.15) + 128.0
    rgb[:, :, 0] += rng.uniform(-15, 15)
    rgb[:, :, 1] += rng.uniform(-10, 10)
    rgb[:, :, 2] += rng.uniform(-15, 15)
    if rng.random() < 0.45:
        h, w = alpha.shape
        low = rng.uniform(0.35, 0.85)
        if rng.random() < 0.5:
            grad = np.linspace(low, 1.0, w)
            if rng.random() < 0.5:
                grad = grad[::-1]
            grad = np.tile(grad, (h, 1))
        else:
            grad = np.linspace(low, 1.0, h)
            if rng.random() < 0.5:
                grad = grad[::-1]
            grad = np.tile(grad.reshape(-1, 1), (1, w))
        rgb = rgb * grad[:, :, None]
    rgb = np.clip(rgb, 0, 255).astype(np.uint8)
    out = np.dstack([rgb, alpha.astype(np.uint8)])
    return Image.fromarray(out, 'RGBA')

def synthesize(pv_train_, backgrounds, num_per_leaf, max_total):
    if SYNTH_INDEX.exists():
        cached = json.load(open(SYNTH_INDEX))
        recovered = [Sample(Path(e['path']), e['label'], 'synthetic') for e in cached]
        if len(recovered) >= 0.9 * max_total:
            print(f' cached synthetic: {len(recovered)}')
            return recovered
    if not backgrounds:
        print(' no backgrounds available, skipping')
        return []
```

```

rng = random.Random(SEED + 17)
by_class = defaultdict(list)
for s in pv_train_:
    by_class[s.label].append(s)
leaves_per_class = max(4, (max_total // max(1, len(by_class))) // num_per_leaf)
print(f' target: {leaves_per_class} leaves x {num_per_leaf} bgs x {len(by_class)}')
out, total = [], 0
for label, items in by_class.items():
    rng.shuffle(items)
    used = 0
    for source in items:
        if used >= leaves_per_class or total >= max_total:
            break
        try:
            img = Image.open(source.path).convert('RGB')
            cutout = segment_leaf(img)
            if cutout is None:
                continue
            alpha = np.asarray(cutout)[:, :, 3]
            ys, xs = np.where(alpha > 32)
            if ys.size == 0:
                continue
            cutout = cutout.crop((int(xs.min()), int(ys.min()), int(xs.max()) + 1, int(ys.max()) + 1))
        except Exception:
            continue
        for variant in range(num_per_leaf):
            if total >= max_total:
                break
            try:
                leaf = augment_leaf(cutout, rng)
                bg = Image.open(rng.choice(backgrounds)).convert('RGB').copy()
                bw, bh = bg.size
                target_side = int(min(bw, bh) * rng.uniform(0.50, 0.95))
                lw, lh = leaf.size
                scale = target_side / max(lw, lh)
                leaf = leaf.resize((max(8, int(lw * scale)), max(8, int(lh * scale))))
                leaf = leaf.rotate(rng.uniform(-45, 45), expand=True, resample=Image.Resample.LANCZOS)
                lw, lh = leaf.size
                if lw >= bw or lh >= bh:
                    continue
                bg.paste(leaf, (rng.randint(0, bw - lw), rng.randint(0, bh - lh)), Image.Resample.LANCZOS)
                out_path = SYNTH_LEAVES_DIR / f'synth_c{label:02d}_{source.path.split("/")[-1]}.png'
                bg.save(out_path, quality=85)
                out.append(Sample(out_path, label, 'synthetic'))
                total += 1
            except Exception:
                continue
        used += 1
    if total and total % 2000 < num_per_leaf:
        print(f' progress: {total}/{max_total}')
json.dump([{'path': str(s.path), 'label': s.label} for s in out], open(SYNTH_IN_JSON, 'w'))
print(f' generated {len(out)} synthetic samples')
return out

```

```

synth_samples = synthesize(pv_train, bg_pool, CFG.num_synth_per_leaf, CFG.max_synth_per_class)
print(f'Synthetic-field: {len(synth_samples)}')

```

target: 219 leaves x 3 bgs x 38 classes
progress: 14002/25000
generated 15925 synthetic samples
Synthetic-field: 15925

Training pool + sampler

```
In [17]: train_samples = pv_train + synth_samples + pd_train + pp_train
val_samples = pv_val + pd_val + pp_val

test_sets = {
    'plantvillage_test': pv_test,
    'plantdoc_test': pd_test,
}
if pp_test:
    test_sets['plantpath_test'] = pp_test

calibration_samples = (pd_val + pp_val) if (len(pd_val) + len(pp_val)) >= 100 else

print(f'TRAIN total: {len(train_samples)} (pv={len(pv_train)} synth={len(synth_sa
print(f'VAL total: {len(val_samples)}')
print(f'CALIB total: {len(calibration_samples)}')
for n, s in test_sets.items():
    print(f'TEST {n:20s} {len(s)}')
```

```
TRAIN total: 69674 (pv=43471 synth=15925 pd=1228 pp=9050)
VAL total: 6797
CALIB total: 1380
TEST plantvillage_test 5417
TEST plantdoc_test 250
TEST plantpath_test 1130
```

```
In [18]: SOURCE_WEIGHTS = {
    'plantvillage': CFG.plantvillage_weight,
    'synthetic': CFG.synthetic_weight,
    'plantdoc': CFG.plantdoc_weight,
    'plantpath2021': CFG.plantpath_weight,
}

def weighted_sampler(samples):
    label_counts = Counter(s.label for s in samples)
    weights = [SOURCE_WEIGHTS.get(s.source, 1.0) / max(1, label_counts[s.label]) fo
    return WeightedRandomSampler(weights, num_samples=len(weights), replacement=True
```

Transforms + dataloaders

```
In [19]: IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(CFG.image_size_train, scale=(0.5, 1.0), ratio=(0.7
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.15),
```

```

        transforms.RandAugment(num_ops=2, magnitude=9),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
        transforms.RandomErasing(p=0.25, scale=(0.02, 0.2)),
    ])
    eval_transform = transforms.Compose([
        transforms.Resize(CFG.image_size_eval + 32),
        transforms.CenterCrop(CFG.image_size_eval),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

    class LeafDataset(Dataset):
        def __init__(self, samples, tfm):
            self.samples = samples
            self.tfm = tfm
        def __len__(self):
            return len(self.samples)
        def __getitem__(self, i):
            s = self.samples[i]
            try:
                img = Image.open(s.path).convert('RGB')
            except Exception:
                img = Image.new('RGB', (CFG.image_size_train, CFG.image_size_train))
            return self.tfm(img), s.label

    train_loader = DataLoader(LeafDataset(train_samples, train_transform), batch_size=CFG.batch_size,
                              sampler=weighted_sampler(train_samples), num_workers=CFG.num_workers,
                              pin_memory=torch.cuda.is_available(), drop_last=True,
                              persistent_workers=CFG.num_workers > 0)
    val_loader = DataLoader(LeafDataset(val_samples, eval_transform), batch_size=CFG.batch_size,
                             shuffle=False, num_workers=CFG.num_workers,
                             pin_memory=torch.cuda.is_available(),
                             persistent_workers=CFG.num_workers > 0)
    calibration_loader = DataLoader(LeafDataset(calibration_samples, eval_transform), batch_size=CFG.batch_size,
                                     shuffle=False, num_workers=CFG.num_workers,
                                     pin_memory=torch.cuda.is_available())
    test_loaders = {n: DataLoader(LeafDataset(s, eval_transform), batch_size=CFG.batch_size,
                                     shuffle=False, num_workers=CFG.num_workers,
                                     pin_memory=torch.cuda.is_available())
                     for n, s in test_sets.items() if s}

    x, y = next(iter(train_loader))
    print(f'Batch: {tuple(x.shape)} labels [{y.min().item()}, {y.max().item()}] train')

```

Batch: (32, 3, 224, 224) labels [0, 36] train batches/epoch=2177

MixUp + CutMix

```

In [20]: def mixup(x, y, alpha):
            lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
            idx = torch.randperm(x.size(0), device=x.device)
            return lam * x + (1 - lam) * x[idx], y, y[idx], lam

        def cutmix(x, y, alpha):

```

```

lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
idx = torch.randperm(x.size(0), device=x.device)
_, _, H, W = x.size()
cr = math.sqrt(1 - lam)
cw, ch = int(W * cr), int(H * cr)
cx, cy = int(np.random.randint(W)), int(np.random.randint(H))
x1, y1 = max(cx - cw // 2, 0), max(cy - ch // 2, 0)
x2, y2 = min(cx + cw // 2, W), min(cy + ch // 2, H)
x = x.clone()
x[:, :, y1:y2, x1:x2] = x[idx, :, y1:y2, x1:x2]
return x, y, y[idx], 1 - ((x2 - x1) * (y2 - y1) / (W * H))

def mix_loss(crit, logits, ya, yb, lam):
    return lam * crit(logits, ya) + (1 - lam) * crit(logits, yb)

```

LoRA + DINOv2 with LoRA-adapted attention layers

`LoRALinear` wraps a frozen `nn.Linear` and adds two trainable low-rank matrices A ($r \times \text{in}$) and B ($\text{out} \times r$). Output = $\text{base}(x) + (B @ A @ x) * (\alpha / r)$. With $r=8$, this adds ~12K params per qkv layer and ~6K per proj — total ~220K across DINOv2's 12 blocks.

```

In [21]: class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r: int = 8, alpha: float = 16.0):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False
        in_f, out_f = base.in_features, base.out_features
        self.lora_A = nn.Parameter(torch.empty(r, in_f))
        self.lora_B = nn.Parameter(torch.zeros(out_f, r))
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        self.scaling = alpha / r
    def forward(self, x):
        return self.base(x) + (x @ self.lora_A.t()) @ self.lora_B.t() * self.scaling

def apply_lora_to_dinov2(backbone, r=8, alpha=16.0):
    n_wrapped = 0
    for block in backbone.blocks:
        block.attn.qkv = LoRALinear(block.attn.qkv, r=r, alpha=alpha)
        block.attn.proj = LoRALinear(block.attn.proj, r=r, alpha=alpha)
        n_wrapped += 2
    return n_wrapped

```

```

In [22]: class DINOv2LoRAClassifier(nn.Module):
    """DINOv2 ViT-S/14 backbone with LoRA adapters on qkv+proj, plus MLP head."""
    def __init__(self, num_classes: int, lora_r: int = 8, lora_alpha: float = 16.0,
        embed_dim: int = 384, hidden: int = 512, dropout: float = 0.2):
        super().__init__()
        self.backbone = torch.hub.load('facebookresearch/dinov2', 'dinov2_vits14',
        for p in self.backbone.parameters():
            p.requires_grad = False
        n = apply_lora_to_dinov2(self.backbone, r=lora_r, alpha=lora_alpha)
        self.backbone.eval()

```

```

        self.head = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, num_classes),
        )
        print(f' DINOv2 LoRA wrapped {n} linear layers (r={lora_r}, alpha={lora_alpha})')
    def train(self, mode: bool = True):
        super().train(mode)
        self.backbone.eval()
        return self
    def forward(self, x):
        return self.head(self.backbone(x))

def build_model(name):
    if name == 'dinov2_lora_vits14':
        return DINOv2LoRAClassifier(NUM_CLASSES, lora_r=CFG.lora_r, lora_alpha=CFG.lora_alpha)
    if name == 'efficientnet_v2_s':
        m = models.efficientnet_v2_s(weights=models.EfficientNet_V2_S_Weights.IMAGE1K)
        m.classifier = nn.Sequential(nn.Dropout(0.3, inplace=True),
                                     nn.Linear(m.classifier[1].in_features, NUM_CLASSES))
        return m
    raise ValueError(name)

for n in ['dinov2_lora_vits14', 'efficientnet_v2_s']:
    m = build_model(n)
    total = sum(p.numel() for p in m.parameters()) / 1e6
    trainable = sum(p.numel() for p in m.parameters() if p.requires_grad) / 1e6
    print(f'{n:25s} total={total:.1f}M trainable={trainable:.2f}M')
del m

```

```

Using cache found in /home/spant/.cache/torch/hub/facebookresearch_dinov2_main
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/swiglu_ffn.py:51: UserWarning: xFormers is not available (SwiGLU)
  warnings.warn("xFormers is not available (SwiGLU)")
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/attention.py:33: UserWarning: xFormers is not available (Attention)
  warnings.warn("xFormers is not available (Attention)")
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/block.py:40: UserWarning: xFormers is not available (Block)
  warnings.warn("xFormers is not available (Block)")
DINOv2 LoRA wrapped 24 linear layers (r=8, alpha=16.0)
dinov2_lora_vits14      total=22.5M trainable=0.44M
efficientnet_v2_s      total=20.2M trainable=20.23M

```

EMA wrapper + scheduler + eval helpers + training loop

```

In [23]: class EMA:
    def __init__(self, model, decay=0.999):
        self.decay = decay
        self.shadow = {k: v.detach().clone() for k, v in model.state_dict().items()}
        @torch.no_grad()

```

```

def update(self, model):
    for k, v in model.state_dict().items():
        if v.dtype.is_floating_point:
            self.shadow[k].mul_(self.decay).add_(v.detach(), alpha=1 - self.decay)
        else:
            self.shadow[k].copy_(v.detach())
def apply_to(self, model):
    model.load_state_dict(self.shadow)

def warmup_cosine(optimizer, warmup_steps, total_steps, min_ratio=0.01):
    def fn(step):
        if step < warmup_steps:
            return (step + 1) / max(1, warmup_steps)
        prog = (step - warmup_steps) / max(1, total_steps - warmup_steps)
        return max(min_ratio, 0.5 * (1 + math.cos(math.pi * min(1.0, prog))))
    return torch.optim.lr_scheduler.LambdaLR(optimizer, fn)

@torch.no_grad()
def collect_logits(model, loader):
    model.eval()
    L, Y = [], []
    for x, y in loader:
        L.append(model(x.to(device, non_blocking=True)).cpu())
        Y.append(y)
    return torch.cat(L), torch.cat(Y)

def evaluate_logits(logits, labels, T=1.0):
    p = torch.softmax(logits / T, dim=1)
    pred = p.argmax(1)
    return {
        'accuracy': float((pred == labels).float().mean()),
        'macro_f1': float(f1_score(labels.numpy(), pred.numpy(), average='macro', z
    }

```

```

In [24]: def train(name, lr=None):
    save_dir = ARTIFACT_DIR / name
    save_dir.mkdir(parents=True, exist_ok=True)
    model = build_model(name).to(device)
    crit = nn.CrossEntropyLoss(label_smoothing=CFG.label_smoothing)
    trainable = [p for p in model.parameters() if p.requires_grad]
    use_lr = lr if lr is not None else CFG.learning_rate
    opt = torch.optim.AdamW(trainable, lr=use_lr, weight_decay=CFG.weight_decay)
    total_steps = CFG.epochs * len(train_loader)
    sched = warmup_cosine(opt, CFG.warmup_epochs * len(train_loader), total_steps)
    scaler = torch.amp.GradScaler('cuda', enabled=device.type == 'cuda')
    ema = EMA(model, decay=CFG.ema_decay)
    best_f1, best_state, history = -1.0, None, []
    rng = random.Random(SEED + 41)
    ema_model = build_model(name).to(device)
    for epoch in range(1, CFG.epochs + 1):
        model.train()
        run_loss, run_n = 0.0, 0
        for x, y in train_loader:
            x = x.to(device, non_blocking=True); y = y.to(device, non_blocking=True)
            r = rng.random()
            if r < CFG.mix_prob / 2:

```

```

        xm, ya, yb, lam = mixup(x, y, CFG.mixup_alpha); mixed = True
    elif r < CFG.mix_prob:
        xm, ya, yb, lam = cutmix(x, y, CFG.cutmix_alpha); mixed = True
    else:
        xm, ya, yb, lam, mixed = x, y, y, 1.0, False
    opt.zero_grad(set_to_none=True)
    with torch.autocast(device_type=device.type, enabled=device.type == 'cuda'):
        logits = model(xm)
        loss = mix_loss(crit, logits, ya, yb, lam) if mixed else crit(logits, ya)
        scaler.scale(loss).backward(); scaler.step(opt); scaler.update(); scheduler.step()
        ema.update(model)
    run_loss += loss.item() * x.size(0); run_n += x.size(0)
train_loss = run_loss / max(1, run_n)
ema.apply_to(model)
vl, vy = collect_logits(ema_model, val_loader)
vm = evaluate_logits(vl, vy)
row = {'epoch': epoch, 'train_loss': round(train_loss, 4),
        'val_acc': round(vm['accuracy'], 4), 'val_f1': round(vm['macro_f1'], 4),
        'lr': round(scheduler.get_last_lr()[0], 6)}
history.append(row); print(json.dumps(row))
torch.save({'state_dict': ema_model.state_dict(), 'epoch': epoch}, save_dir)
if vm['macro_f1'] > best_f1:
    best_f1 = vm['macro_f1']
    best_state = {k: v.detach().cpu().clone() for k, v in ema_model.state_dict().items()}
    torch.save({'state_dict': best_state, 'epoch': epoch, 'val_f1': best_f1}, save_dir)
model.load_state_dict(best_state)
return model, history, best_f1

```

Train LoRA-adapted DINOv2

```

In [25]: dino_model, dino_history, dino_best = train('dinov2_lora_vits14', lr=CFG.dinov2_lr)
print(f'DINOv2-LoRA best val_f1 = {dino_best:.4f}')

```

Using cache found in /home/spant/.cache/torch/hub/facebookresearch_dinov2_main
DINOv2 LoRA wrapped 24 linear layers (r=8, alpha=16.0)

Using cache found in /home/spant/.cache/torch/hub/facebookresearch_dinov2_main

DINOv2 LoRA wrapped 24 linear layers (r=8, alpha=16.0)

```
{
  "epoch": 1, "train_loss": 1.9533, "val_acc": 0.952, "val_f1": 0.943, "lr": 0.000167
}
{
  "epoch": 2, "train_loss": 1.2568, "val_acc": 0.974, "val_f1": 0.9702, "lr": 0.000333
}
{
  "epoch": 3, "train_loss": 1.2194, "val_acc": 0.979, "val_f1": 0.9755, "lr": 0.0005
}
{
  "epoch": 4, "train_loss": 1.2017, "val_acc": 0.9812, "val_f1": 0.9783, "lr": 0.000497
}
{
  "epoch": 5, "train_loss": 1.1439, "val_acc": 0.9806, "val_f1": 0.9778, "lr": 0.00049
}
{
  "epoch": 6, "train_loss": 1.14, "val_acc": 0.9834, "val_f1": 0.9792, "lr": 0.000477
}
{
  "epoch": 7, "train_loss": 1.1166, "val_acc": 0.9831, "val_f1": 0.9799, "lr": 0.00046
}
{
  "epoch": 8, "train_loss": 1.1009, "val_acc": 0.9841, "val_f1": 0.9809, "lr": 0.000439
}
{
  "epoch": 9, "train_loss": 1.1021, "val_acc": 0.9838, "val_f1": 0.9801, "lr": 0.000414
}
{
  "epoch": 10, "train_loss": 1.0833, "val_acc": 0.9818, "val_f1": 0.9775, "lr": 0.000385
}
{
  "epoch": 11, "train_loss": 1.0671, "val_acc": 0.9844, "val_f1": 0.9809, "lr": 0.000354
}
{
  "epoch": 12, "train_loss": 1.0583, "val_acc": 0.984, "val_f1": 0.9806, "lr": 0.00032
}
{
  "epoch": 13, "train_loss": 1.0462, "val_acc": 0.9847, "val_f1": 0.9813, "lr": 0.000286
}
{
  "epoch": 14, "train_loss": 1.0492, "val_acc": 0.9837, "val_f1": 0.9788, "lr": 0.00025
}
{
  "epoch": 15, "train_loss": 1.0319, "val_acc": 0.9844, "val_f1": 0.9802, "lr": 0.000214
}
{
  "epoch": 16, "train_loss": 1.0064, "val_acc": 0.9846, "val_f1": 0.9806, "lr": 0.00018
}
{
  "epoch": 17, "train_loss": 1.0119, "val_acc": 0.9846, "val_f1": 0.9803, "lr": 0.000146
}
{
  "epoch": 18, "train_loss": 1.0129, "val_acc": 0.9853, "val_f1": 0.9802, "lr": 0.000115
}
{
  "epoch": 19, "train_loss": 0.9958, "val_acc": 0.985, "val_f1": 0.9798, "lr": 8.6e-05
}
{
  "epoch": 20, "train_loss": 0.9762, "val_acc": 0.9851, "val_f1": 0.9797, "lr": 6.1e-05
}
{
  "epoch": 21, "train_loss": 0.9895, "val_acc": 0.9853, "val_f1": 0.9801, "lr": 4e-05
}
{
  "epoch": 22, "train_loss": 0.9738, "val_acc": 0.9857, "val_f1": 0.9804, "lr": 2.3e-05
}
{
  "epoch": 23, "train_loss": 0.9668, "val_acc": 0.9856, "val_f1": 0.9803, "lr": 1e-05
}
{
  "epoch": 24, "train_loss": 0.9573, "val_acc": 0.9853, "val_f1": 0.9799, "lr": 5e-06
}
{
  "epoch": 25, "train_loss": 0.9782, "val_acc": 0.9851, "val_f1": 0.9798, "lr": 5e-06
}
```

DINOv2-LoRA best val_f1 = 0.9813

Train EfficientNetV2-S

```
In [26]: effnet_model, effnet_history, effnet_best = train('efficientnet_v2_s')
print(f'EfficientNetV2-S best val_f1 = {effnet_best:.4f}')
```

```
{
  "epoch": 1, "train_loss": 1.7935, "val_acc": 0.9648, "val_f1": 0.9538, "lr": 0.000167
}
{
  "epoch": 2, "train_loss": 1.1824, "val_acc": 0.9812, "val_f1": 0.9766, "lr": 0.000333
}
{
  "epoch": 3, "train_loss": 1.1891, "val_acc": 0.9801, "val_f1": 0.9756, "lr": 0.0005
}
{
  "epoch": 4, "train_loss": 1.1732, "val_acc": 0.9797, "val_f1": 0.9761, "lr": 0.000497
}
{
  "epoch": 5, "train_loss": 1.1151, "val_acc": 0.9819, "val_f1": 0.9765, "lr": 0.00049
}
{
  "epoch": 6, "train_loss": 1.1188, "val_acc": 0.9815, "val_f1": 0.9775, "lr": 0.000477
}
{
  "epoch": 7, "train_loss": 1.0636, "val_acc": 0.9821, "val_f1": 0.9785, "lr": 0.00046
}
{
  "epoch": 8, "train_loss": 1.0713, "val_acc": 0.9832, "val_f1": 0.9793, "lr": 0.000439
}
{
  "epoch": 9, "train_loss": 1.0532, "val_acc": 0.9826, "val_f1": 0.9777, "lr": 0.000414
}
{
  "epoch": 10, "train_loss": 1.0383, "val_acc": 0.9823, "val_f1": 0.9779, "lr": 0.000385
}
{
  "epoch": 11, "train_loss": 1.0108, "val_acc": 0.9846, "val_f1": 0.9804, "lr": 0.000354
}
{
  "epoch": 12, "train_loss": 1.0042, "val_acc": 0.9857, "val_f1": 0.9816, "lr": 0.00032
}
{
  "epoch": 13, "train_loss": 0.9794, "val_acc": 0.9835, "val_f1": 0.9787, "lr": 0.000286
}
{
  "epoch": 14, "train_loss": 0.979, "val_acc": 0.9846, "val_f1": 0.9806, "lr": 0.00025
}
{
  "epoch": 15, "train_loss": 0.9479, "val_acc": 0.9844, "val_f1": 0.9803, "lr": 0.000214
}
{
  "epoch": 16, "train_loss": 0.9478, "val_acc": 0.9846, "val_f1": 0.9796, "lr": 0.00018
}
{
  "epoch": 17, "train_loss": 0.9453, "val_acc": 0.9859, "val_f1": 0.9819, "lr": 0.000146
}
{
  "epoch": 18, "train_loss": 0.9462, "val_acc": 0.9862, "val_f1": 0.9824, "lr": 0.000115
}
{
  "epoch": 19, "train_loss": 0.9379, "val_acc": 0.9857, "val_f1": 0.9814, "lr": 8.6e-05
}
{
  "epoch": 20, "train_loss": 0.9176, "val_acc": 0.9865, "val_f1": 0.9817, "lr": 6.1e-05
}
{
  "epoch": 21, "train_loss": 0.93, "val_acc": 0.9859, "val_f1": 0.9802, "lr": 4e-05
}
{
  "epoch": 22, "train_loss": 0.9187, "val_acc": 0.9871, "val_f1": 0.982, "lr": 2.3e-05
}
{
  "epoch": 23, "train_loss": 0.9072, "val_acc": 0.9865, "val_f1": 0.981, "lr": 1e-05
}
{
  "epoch": 24, "train_loss": 0.9106, "val_acc": 0.9868, "val_f1": 0.9815, "lr": 5e-06
}
{
  "epoch": 25, "train_loss": 0.9158, "val_acc": 0.9869, "val_f1": 0.9817, "lr": 5e-06
}
EfficientNetV2-S best val_f1 = 0.9824
```

Calibration (target precision 0.80)

```

In [27]: def fit_temperature(logits, labels):
    lo, la = logits.to(device), labels.to(device)
    t = torch.ones(1, device=device, requires_grad=True)
    opt = torch.optim.LBFGS([t], lr=0.01, max_iter=50)
    crit = nn.CrossEntropyLoss()
    def closure():
        opt.zero_grad()
        loss = crit(lo / t.clamp(0.5, 5.0), la)
        loss.backward()
        return loss
    opt.step(closure)
    return float(t.detach().clamp(0.5, 5.0).item())

def uncertainty(logits, T):
    p = torch.softmax(logits / T, dim=1)
    top2 = p.topk(2, dim=1).values
    return (p.argmax(1), top2[:, 0], top2[:, 0] - top2[:, 1],
            -(p * torch.log(p.clamp_min(1e-8))).sum(1) / math.log(NUM_CLASSES))

def search_thresholds(logits, labels, T, target):
    pred, mp, mg, en = uncertainty(logits, T)
    correct = pred == labels
    best = None
    for c in np.linspace(0.35, 0.95, 13):
        for m in np.linspace(0.05, 0.45, 9):
            for e in np.linspace(0.25, 0.85, 13):
                mask = (mp >= c) & (mg >= m) & (en <= e)
                if mask.sum() < 20:
                    continue
                prec = float(correct[mask].float().mean())
                cov = float(mask.float().mean())
                if prec >= target and (best is None or cov > best['confident_coverage']):
                    best = {'max_prob_min': round(float(c), 4), 'margin_min': round(
                        float(m), 4), 'entropy_max': round(float(e), 4),
                        'confident_precision': round(prec, 4), 'confident_coverage': round(cov, 4)}
    return best or {'max_prob_min': 0.60, 'margin_min': 0.15, 'entropy_max': 0.65,
                    'confident_precision': None, 'confident_coverage': None}

def calibrate(model, name):
    vl, vy = collect_logits(model, val_loader)
    cl, cy = collect_logits(model, calibration_loader)
    T = fit_temperature(vl, vy)
    thr = search_thresholds(cl, cy, T, CFG.target_confident_precision)
    print(f'{name}: T={T:.3f} thresholds={thr}')
    return T, thr

dino_T, dino_thr = calibrate(dino_model, 'dinov2_lora_vits14')
effnet_T, effnet_thr = calibrate(effnet_model, 'efficientnet_v2_s')

```

```

dinov2_lora_vits14: T=0.879 thresholds={'max_prob_min': 0.35, 'margin_min': 0.05,
'entropy_max': 0.6, 'confident_precision': 0.9428, 'confident_coverage': 0.9877}
efficientnet_v2_s: T=0.878 thresholds={'max_prob_min': 0.35, 'margin_min': 0.05, 'e
ntropy_max': 0.85, 'confident_precision': 0.9388, 'confident_coverage': 0.9942}

```

Per-source evaluation

```
In [28]: def evaluate_all(model, T, name):
    print(f'\n=== {name} (T={T:.3f}) ===')
    out = {}
    for src, ldr in test_loaders.items():
        lg, lb = collect_logits(model, ldr)
        m = evaluate_logits(lg, lb, T)
        m['n'] = int(lb.size(0))
        out[src] = m
        print(f' {src:25s} n={m["n"]:>5d} acc={m["accuracy"]:.4f} f1={m["macro_f1"]:.4f}')
    return out

dino_per_source = evaluate_all(dino_model, dino_T, 'dinov2_lora_vits14')
effnet_per_source = evaluate_all(effnet_model, effnet_T, 'efficientnet_v2_s')

=== dinov2_lora_vits14 (T=0.879) ===
plantvillage_test      n= 5417  acc=0.9956  f1=0.9942
plantdoc_test          n=   250  acc=0.7400  f1=0.6987
plantpath_test         n=  1130  acc=0.9858  f1=0.9860

=== efficientnet_v2_s (T=0.878) ===
plantvillage_test      n= 5417  acc=0.9974  f1=0.9964
plantdoc_test          n=   250  acc=0.6960  f1=0.6801
plantpath_test         n=  1130  acc=0.9903  f1=0.9906
```

Ensemble (averaged softmax)

```
In [29]: @torch.no_grad()
def collect_softmax(model, loader, T):
    model.eval()
    P, Y = [], []
    for x, y in loader:
        logits = model(x.to(device, non_blocking=True))
        P.append(torch.softmax(logits / T, dim=1).cpu())
        Y.append(y)
    return torch.cat(P), torch.cat(Y)

def evaluate_ensemble(loader):
    pd_, yd = collect_softmax(dino_model, loader, dino_T)
    pe_, ye = collect_softmax(effnet_model, loader, effnet_T)
    assert torch.equal(yd, ye)
    probs = (pd_ + pe_) / 2.0
    pred = probs.argmax(1)
    return {
        'accuracy': float((pred == yd).float().mean()),
        'macro_f1': float(f1_score(yd.numpy(), pred.numpy(), average='macro', zero_division=0)),
        'n': int(yd.size(0)),
    }

print(f'\n=== ENSEMBLE (DINOv2-LoRA + EffNet) ===')
ensemble_per_source = {}
for src, ldr in test_loaders.items():
    m = evaluate_ensemble(ldr)
```

```
ensemble_per_source[src] = m
print(f' {src:25s} n={m["n"]:>5d} acc={m["accuracy"]:.4f} f1={m["macro_f1"]:
```

```
=== ENSEMBLE (DINOv2-LoRA + EffNet) ===
plantvillage_test      n= 5417  acc=0.9972  f1=0.9960
plantdoc_test          n=  250  acc=0.7560  f1=0.7476
plantpath_test         n= 1130  acc=0.9894  f1=0.9899
```

TTA + ensemble (final number)

```
In [30]: tta_base = transforms.Compose([
    transforms.Resize(CFG.image_size_eval + 32), transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

def five_crop_flips(t, sz):
    _, h, w = t.shape
    crops = []
    cy, cx = (h - sz) // 2, (w - sz) // 2
    for (y, x) in [(0, 0), (0, w - sz), (h - sz, 0), (h - sz, w - sz), (cy, cx)]:
        c = t[:, y:y+sz, x:x+sz]
        crops.append(c)
        crops.append(torch.flip(c, dims=[2]))
    return torch.stack(crops)

@torch.no_grad()
def predict_tta(model, image_path, T, crop_size=CFG.image_size_eval):
    img = Image.open(image_path).convert('RGB')
    crops = five_crop_flips(tta_base(img), crop_size).to(device)
    model.eval()
    return torch.softmax(model(crops) / T, dim=1).mean(0).cpu()

@torch.no_grad()
def predict_tta_ensemble(image_path):
    pd_ = predict_tta(dino_model, image_path, dino_T, CFG.image_size_eval)
    pe_ = predict_tta(effnet_model, image_path, effnet_T, CFG.image_size_eval)
    return (pd_ + pe_) / 2.0

def evaluate_tta_ensemble(samples, max_n=400):
    rng = random.Random(SEED + 71)
    sub = samples if len(samples) <= max_n else rng.sample(samples, max_n)
    correct = sum(1 for s in sub if int(predict_tta_ensemble(s.path).argmax()) == s)
    return correct / max(1, len(sub)), len(sub)

print('\n--- TTA + ENSEMBLE ---')
tta_ensemble_per_source = {}
for tname, ts in test_sets.items():
    if ts:
        acc, n = evaluate_tta_ensemble(ts)
        tta_ensemble_per_source[tname] = {'accuracy': acc, 'n': n}
        print(f' {tname:25s} TTA+ensemble acc={acc:.4f} (n={n})')
```

--- TTA + ENSEMBLE ---

plantvillage_test	TTA+ensemble acc=0.9975 (n=400)
plantdoc_test	TTA+ensemble acc=0.7440 (n=250)
plantpath_test	TTA+ensemble acc=0.9950 (n=400)

Save bundles + report

```
In [31]: def save_bundle(name, model, history, T, thr, per_source):
w = ARTIFACT_DIR / f'{name}_cropscan_v5.pth'
b = ARTIFACT_DIR / f'{name}_cropscan_v5_bundle.pt'
torch.save(model.state_dict(), w)
torch.save({
    'state_dict': model.state_dict(),
    'temperature': T,
    'thresholds': thr,
    'class_names': CLASS_NAMES,
    'version': 'cropscan-v5',
    'architecture': name,
    'history': history,
    'per_source_test_metrics': per_source,
    'lora_r': CFG.lora_r,
    'lora_alpha': CFG.lora_alpha,
}, b)
print(f' {w}')
print(f' {b}')

save_bundle('dinov2_lora_vits14', dino_model, dino_history, dino_T, dino_thr,
save_bundle('efficientnet_v2_s', effnet_model, effnet_history, effnet_T, effnet_th

report = {
    'class_names': CLASS_NAMES,
    'data_sizes': {
        'train_total': len(train_samples),
        'pv_train': len(pv_train), 'synthetic': len(synth_samples),
        'pd_train': len(pd_train), 'pp_train': len(pp_train),
        'val_total': len(val_samples), 'calibration_total': len(calibration_samples
    },
    'test_sizes': {k: len(v) for k, v in test_sets.items()},
    'dinov2_lora_vits14': {'history': dino_history, 'temperature': dino_T, 'thr
    'efficientnet_v2_s': {'history': effnet_history, 'temperature': effnet_T, 'thr
    'ensemble': {'per_source': ensemble_per_source},
    'ensemble_tta': {'per_source': tta_ensemble_per_source},
}
json.dump(report, open(ARTIFACT_DIR / 'training_report.json', 'w'), indent=2)
json.dump(CLASS_NAMES, open(ARTIFACT_DIR / 'labels.json', 'w'), indent=2)
print(f'\nReport: {ARTIFACT_DIR / "training_report.json"}')

artifacts/v5/dinov2_lora_vits14_cropscan_v5.pth
artifacts/v5/dinov2_lora_vits14_cropscan_v5_bundle.pt
artifacts/v5/efficientnet_v2_s_cropscan_v5.pth
artifacts/v5/efficientnet_v2_s_cropscan_v5_bundle.pt
```

Report: artifacts/v5/training_report.json

Visualize ensemble TTA predictions on PlantDoc

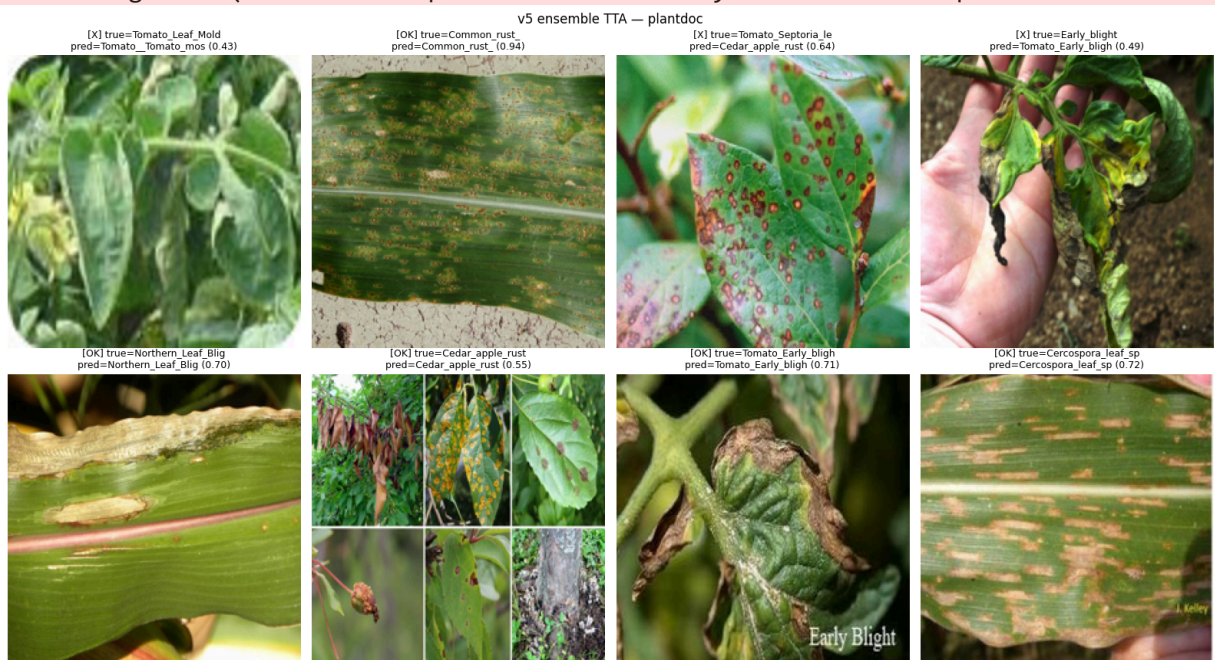
```
In [32]: import matplotlib.pyplot as plt

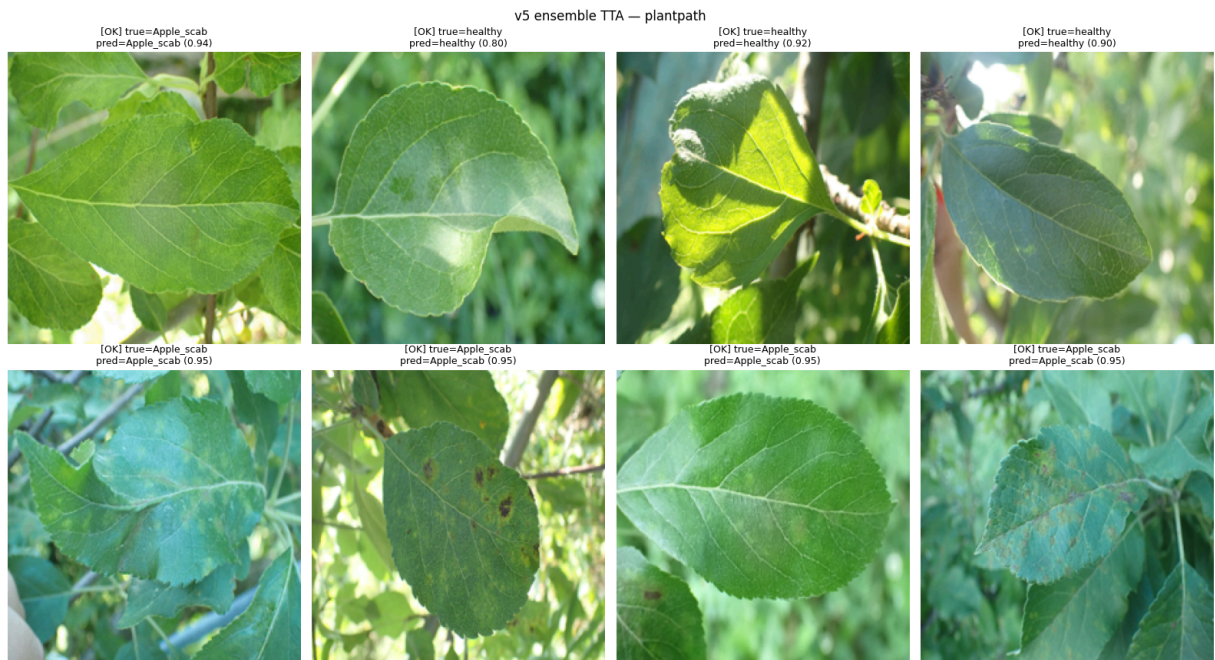
def visualize_ensemble(samples, name='ensemble_pd', n=8):
    if not samples:
        return
    rng = random.Random(SEED + 101)
    picks = rng.sample(samples, min(n, len(samples)))
    fig, axes = plt.subplots(2, 4, figsize=(16, 9))
    for ax, s in zip(axes.flat, picks):
        probs = predict_tta_ensemble(s.path)
        top = int(probs.argmax())
        ax.imshow(Image.open(s.path).convert('RGB').resize((224, 224)))
        ax.axis('off')
        mark = 'OK' if top == s.label else 'X'
        true = CLASS_NAMES[s.label].split('__')[-1][:18]
        pred = CLASS_NAMES[top].split('__')[-1][:18]
        ax.set_title(f'[{mark}] true={true}\npred={pred} ({probs[top]:.2f})', fonts
plt.suptitle(f'v5 ensemble TTA - {name}')
plt.tight_layout()
plt.savefig(ARTIFACT_DIR / f'preview_{name}.png', dpi=100, bbox_inches='tight')
plt.show()

visualize_ensemble(pd_test, 'plantdoc')
if pp_test:
    visualize_ensemble(pp_test, 'plantpath')
```

/opt/jupyterhub/jupyter_env/lib/python3.12/site-packages/matplotlib/projections/_in
it__.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple versio
ns of Matplotlib being installed (e.g. as a system package and as a pip package). As
a result, the 3D projection is not available.

warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "





Backend integration

v5 changes vs v4: DINOv2 now has LoRA adapters baked into its state_dict, so the backend builder must apply LoRA before loading weights.

1. Copy artifacts to `backend/models/` :

- `dinov2_lora_vits14_cropscan_v5.pth`
- `efficientnet_v2_s_cropscan_v5.pth`
- matching `*_bundle.pt`
- `training_report.json` , `labels.json`

2. Add LoRA + DINOv2 builder to `backend/app/inference.py` :

```
import math, torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, base, r=8, alpha=16.0):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False
        in_f, out_f = base.in_features, base.out_features
        self.lora_A = nn.Parameter(torch.empty(r, in_f))
        self.lora_B = nn.Parameter(torch.zeros(out_f, r))
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        self.scaling = alpha / r
    def forward(self, x):
        return self.base(x) + (x @ self.lora_A.t()) @ self.lora_B.t() * self.scaling
```



```

class DINOv2LoRAClassifier(nn.Module):
    def __init__(self, num_classes, r=8, alpha=16.0, embed_dim=384,
hidden=512, dropout=0.2):
        super().__init__()
        self.backbone = torch.hub.load('facebookresearch/dinov2',
'dinov2_vits14', skip_validation=True)
        for p in self.backbone.parameters():
            p.requires_grad = False
        for block in self.backbone.blocks:
            block.attn.qkv = LoRALinear(block.attn.qkv, r=r,
alpha=alpha)
            block.attn.proj = LoRALinear(block.attn.proj, r=r,
alpha=alpha)
        self.backbone.eval()
        self.head = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, num_classes),
        )
    def train(self, mode=True):
        super().train(mode)
        self.backbone.eval()
        return self
    def forward(self, x):
        return self.head(self.backbone(x))

def _build_dinov2_lora_vits14():
    return DINOv2LoRAClassifier(len(CLASS_NAMES))

```

3. Eval resolution is 280: `Resize(312) + CenterCrop(280)` .
4. At inference: pass image through DINOv2 and EffNet, divide each by its own temperature, average the two softmax distributions, then argmax. Wire abstention thresholds from `training_report.json` (target 80% confident-precision).

In []: